Quick Reference Handbook

DEVOPS ENGINEERING MASTERY — Volume 08, Companion Volume. First Edition, 9 November 2026. Bengali-English glossary (150+ terms), Terraform command and HCL references, plan/apply and import workflows, production checklists, troubleshooting decision trees, the competency tracking sheet and official references — keep this beside your terminal. Terraform 1.9+, AWS, Azure, GCP, IBM Cloud.

শব্দকোষ ১ • Glossary 1

শব্দকোষ — প্রথম অংশ

শব্দকোষ — প্রথম অংশ

প্রতিটি শব্দের বাংলা ব্যাখ্যার সাথে ব্যবহারের প্রসঙ্গ দেওয়া আছে।

Infrastructure as Code (IaC): অবকাঠামোকে কোড হিসেবে সংজ্ঞায়িত ও পরিচালনা করার পদ্ধতি; Git-এ সংস্করণ নিয়ন্ত্রণ, পর্যালোচনা ও পুনরাবৃত্তি সম্ভব হয়।

Declarative: শেষ অবস্থা ঘোষণা করার পদ্ধতি; কীভাবে পৌঁছাতে হবে তা টুল ঠিক করে — Terraform এই পদ্ধতিতে কাজ করে।

Imperative: ধাপে ধাপে কী করতে হবে তা লেখার পদ্ধতি; যেমন Bash স্ক্রিপ্টে সার্ভার তৈরি করে তারপর কনফিগার করা।

Terraform: HashiCorp-এর IaC টুল; HCL-এ লেখা কনফিগারেশন থেকে বিভিন্ন ক্লাউডে অবকাঠামো তৈরি ও পরিচালনা করে।

HCL (HashiCorp Configuration Language): Terraform-এর কনফিগারেশন ভাষা; মানুষের পড়ার মতো, ব্লক ও argument দিয়ে কাঠামো লেখা হয়।

Provider: Terraform-কে নির্দিষ্ট প্ল্যাটফর্মের (AWS, Azure, GCP) API-এর সাথে যুক্ত করা প্লাগইন।

Resource: Terraform পরিচালনা করে এমন অবকাঠামো উপাদান; যেমন aws_instance বা azurerm_virtual_machine।

Data Source: বিদ্যমান অবকাঠামো বা বাইরের তথ্য পড়ার ব্লক; Terraform তা তৈরি করে না, শুধু ব্যবহার করে।

State: Terraform-এর রেকর্ড যেখানে বর্তমান অবকাঠামোর অবস্থা ও কনফিগারেশন-mapping থাকে; plan-এর ভিত্তি।

State File (.tfstate): state সংরক্ষণ করা JSON ফাইল; এতে resource ID, attribute ও dependency থাকে এবং প্রায়ই sensitive মানও থাকে।

Backend: state কোথায় সংরক্ষণ হবে ও locking কীভাবে হবে তা নির্ধারণ করে; যেমন s3, azurerm, gcs, local।

Remote Backend: দূরের সংরক্ষণে (যেমন S3) state রাখার ব্যবস্থা; দলগত কাজ ও locking সম্ভব করে।

State Locking: একসাথে একাধিক apply যাতে state নষ্ট না করে, সেই সুরক্ষা; S3-তে DynamoDB, Azure-তে Blob lease ব্যবহৃত হয়।

Drift: কনফিগারেশনের বাইরে বাস্তব অবকাঠামোতে হাতে বদল; refresh-only plan দিয়ে শনাক্ত করা যায়।

Execution Plan: terraform plan-এর আউটপুট; কী তৈরি, বদল বা ধ্বংস হবে তার পূর্বাভাস।

Apply: পরিকল্পিত পরিবর্তন বাস্তবে প্রয়োগ করার ধাপ; অনুমোদনের পর resource তৈরি/বদল/ধ্বংস হয়।

Destroy: কনফিগারেশনের সব managed resource মুছে দেওয়ার কমান্ড; production-এ অত্যন্ত সতর্কতায় ব্যবহার করতে হয়।

Refresh: বাস্তব অবস্থা পড়ে state আপডেট করার প্রক্রিয়া; বর্তমানে plan -refresh-only দিয়ে করা হয়।

Variable: কনফিগারেশনে ইনপুট মান দেওয়ার উপায়; environment ভেদে মান বদলানো যায়।

Output: apply-এর পর প্রকাশ করা মান; যেমন সার্ভারের IP বা load balancer-এর DNS নাম।

Local Value (locals): একই expression বারবার না লিখে স্থানীয়ভাবে নাম দিয়ে পুনর্ব্যবহার করার উপায়।

Module: পুনর্ব্যবহারযোগ্য Terraform কনফিগারেশনের সংগ্রহ; যেমন VPC module বারবার ব্যবহার করা যায়।

Root Module: Terraform চালানো হয় এমন মূল ডিরেক্টরি; অন্য module এখান থেকে ডাকা হয়।

Child Module: Root module থেকে ডাকা মডিউল; input variable নেয় ও output দেয়।

Registry: HashiCorp-এর ভান্ডার যেখানে যাচাই করা provider ও module পাওয়া যায় (registry.terraform.io)।

Workspace: একই কনফিগারেশনে একাধিক environment-এর আলাদা state রাখার উপায়; যেমন dev, staging, prod।

Count: একই resource-এর একাধিক instance তৈরি করার meta-argument; count.index দিয়ে চেনা যায়।

for_each: map বা set-এর প্রতিটি উপাদানের জন্য instance তৈরি করার meta-argument; স্থিতিশীল key ব্যবহার করে।

depends_on: implicit dependency যথেষ্ট না হলে স্পষ্ট নির্ভরতা ঘোষণা করার meta-argument।

Lifecycle: resource-এর জীবনচক্র নিয়ন্ত্রণ করার ব্লক; prevent_destroy, create_before_destroy, ignore_changes থাকে।

prevent_destroy: lifecycle নিয়ম যা resource ধ্বংসের plan-কে ত্রুটি দিয়ে আটকায়; database-এ ব্যবহৃত হয়।

create_before_destroy: replacement-এর সময় আগে নতুন resource তৈরি করে পরে পুরোনোটি মোছার নিয়ম; downtime কমায়।

ignore_changes: নির্দিষ্ট attribute-এর বাইরের পরিবর্তন plan-এ উপেক্ষা করার lifecycle নিয়ম।

Dynamic Block: for_each দিয়ে পুনরাবৃত্ত nested block তৈরি করার উপায়; যেমন একাধিক ingress rule।

Function: মান গণনা বা রূপান্তর করার built-in উপায়; যেমন merge(), length(), format()।

Expression: HCL-এ মান গণনা করার লেখা; যেমন var.name, aws_vpc.main.id।

Reference: একটি resource বা variable-এর মান অন্য জায়গায় ব্যবহার করা; এতে implicit dependency তৈরি হয়।

Argument: ব্লকের ভেতরে দেওয়া সেটিং; যেমন instance_type = "t3.micro"।

Attribute: resource তৈরি হওয়ার পর তার বৈশিষ্ট্য; যেমন aws_instance.web.public_ip।

Type Constraint: variable-এর মানের ধরন নির্ধারণ করা; যেমন string, list(string), object।

Primitive Type: মৌলিক ধরন; string, number, bool।

Collection Type: একাধিক মানের সংগ্রহ; list, set, map।

Structural Type: নির্দিষ্ট কাঠামোর ধরন; object, tuple।

Sensitive Value: গোপন মান; CLI আউটপুটে গোপন রাখা হয়, তবে state ফাইলে থাকে বলে state সুরক্ষিত রাখতে হয়।

Validation Block: variable-এর মানের শর্ত যাচাই করার ব্লক; শর্ত না মিললে ত্রুটি দেয়।

Import: বিদ্যমান বাস্তব resource-কে Terraform state-এ আনার প্রক্রিয়া।

Import Block: Terraform 1.5+ এ কনফিগারেশনে লিখে import করার উপায়; Git পর্যালোচনায় আনা যায়।

Moved Block: resource-এর ঠিকানা বদলালেও state ধরে রেখে recreation আটকানোর ঘোষণা।

Removed Block: কনফিগারেশন থেকে সরানো resource state থেকেও সরানোর ঘোষণা, বাস্তব resource না মুছে।

Check Block: Terraform 1.5+ এ apply-এর পর assertion দিয়ে অবস্থা যাচাই করার ব্লক।

Ephemeral Value: state-এ সংরক্ষণ না করা অস্থায়ী সংবেদনশীল মান (Terraform 1.10+)।

Write-only Argument: শুধু লেখা যায়, state-এ সংরক্ষণ হয় না এমন argument; যেমন কিছু পাসওয়ার্ড field।

Taint / Replace: resource-কে পরের apply-তে ধ্বংস করে পুনরায় তৈরি করতে চিহ্নিত করা; বর্তমানে -replace flag ব্যবহৃত হয়।

Target (-target): শুধু নির্দিষ্ট resource নিয়ে plan/apply করার flag; নিয়মিত ব্যবহারে drift লুকাতে পারে।

Graph: resource dependency-এর কাঠামো; terraform graph DOT ফরম্যাটে দেখায়।

Plan File: terraform plan -out দিয়ে সংরক্ষণ করা plan; পরে একই plan apply করা যায়।

Lock File (.terraform.lock.hcl): provider-এর নির্দিষ্ট সংস্করণ ও hash লক করা ফাইল; দলের সবাই একই সংস্করণ ব্যবহার করে।

Version Constraint: provider বা Terraform-এর অনুমোদিত সংস্করণের সীমা; যেমন ~> 5.0।

Pessimistic Constraint (~>): ডানদিকের শেষ অংশ বদলাতে দেয় এমন constraint; ~> 5.0 মানে 5.x অনুমোদিত।

tfvars File: variable-এর মান ফাইলে দেওয়ার উপায়; যেমন terraform.tfvars বা prod.tfvars।

TF_VAR_: environment variable থেকে variable-এর মান দেওয়ার উপসর্গ; যেমন TF_VAR_region।

Provisioner: resource তৈরি হওয়ার পর কমান্ড চালানোর উপায়; ভঙ্গুর বলে শেষ উপায় হিসেবে ব্যবহার করা উচিত।

local-exec: Terraform চলা মেশিনে কমান্ড চালানো provisioner।

remote-exec: তৈরি resource-এ SSH/WinRM দিয়ে কমান্ড চালানো provisioner।

null_resource: কোনো ক্লাউড resource তৈরি না করে provisioner চালানোর পুরোনো উপায়; বর্তমানে terraform_data ব্যবহৃত হয়।

terraform_data: null_resource-এর আধুনিক প্রতিস্থাপন; trigger-ভিত্তিক কাজ চালাতে ব্যবহৃত হয়।

AWS Provider: AWS resource পরিচালনার provider; hashicorp/aws।

Azure Provider (azurerm): Azure resource পরিচালনার provider; hashicorp/azurerm।

শব্দকোষ ২ • Glossary 2

শব্দকোষ — দ্বিতীয় অংশ

শব্দকোষ — দ্বিতীয় অংশ

প্রতিটি শব্দের বাংলা ব্যাখ্যার সাথে ব্যবহারের প্রসঙ্গ দেওয়া আছে।

Google Provider: GCP resource পরিচালনার provider; hashicorp/google।

IBM Cloud Provider: IBM Cloud resource পরিচালনার provider; IBM-Cloud/ibm।

AMI: Amazon Machine Image; EC2 instance কোন image থেকে তৈরি হবে তা নির্ধারণ করে।

EC2 Instance: AWS-এর ভার্চুয়াল সার্ভার; Terraform-এ aws_instance দিয়ে তৈরি করা হয়।

S3 Bucket: AWS-এর object storage; Terraform backend-এ state রাখতে ব্যবহৃত হয়।

DynamoDB: AWS-এর NoSQL ডাটাবেস; S3 backend-এ state locking-এর জন্য ব্যবহৃত হয়।

Security Group: AWS-এর ভার্চুয়াল ফায়ারওয়াল; কোন port ও IP থেকে ট্রাফিক আসতে পারবে তা নিয়ন্ত্রণ করে।

VPC: Virtual Private Cloud; ক্লাউডে বিচ্ছিন্ন নেটওয়ার্ক তৈরি করে।

Subnet: VPC-এর ভেতরের ছোট নেটওয়ার্ক অংশ; public/private subnet আলাদা করা হয়।

IAM Role: AWS-এ অনুমতি দেওয়ার পরিচয়; Terraform backend-এ access দিতে ব্যবহৃত হয়।

Resource Group: Azure-এ resource সংগঠিত করার ধারক; একসাথে পরিচালনা সহজ করে।

Blob Storage: Azure-এর object storage; azurerm backend-এ state রাখা হয়।

GCS Bucket: Google Cloud Storage bucket; gcs backend-এ state রাখা হয়।

Policy as Code: নিয়ম কোডে লিখে স্বয়ংক্রিয়ভাবে যাচাই করার পদ্ধতি; যেমন encryption বাধ্যতামূলক করা।

OPA (Open Policy Agent): Rego ভাষায় নীতিমালা লিখে plan যাচাই করার টুল।

Rego: OPA-র নীতিমালা লেখার ভাষা।

Sentinel: HashiCorp-এর Policy as Code টুল; Terraform Cloud-এ ব্যবহৃত হয়।

Checkov: Terraform কনফিগারেশনে নিরাপত্তা ও compliance ত্রুটি খোঁজার static analysis টুল।

TFLint: Terraform কোডে ভুল, deprecated syntax ও provider নিয়ম যাচাই করার linter।

Terratest: Go-ভিত্তিক টেস্ট ফ্রেমওয়ার্ক; আসল অবকাঠামো তৈরি করে যাচাই করে, তারপর মুছে দেয়।

terraform test: Terraform-এর built-in টেস্ট ফ্রেমওয়ার্ক; .tftest.hcl-এ assertion লেখা হয়।

Infracost: plan থেকে আনুমানিক মাসিক খরচ দেখানোর টুল; PR-এ খরচের প্রভাব দেখায়।

CI/CD: Continuous Integration/Continuous Delivery; কোড পরিবর্তনে স্বয়ংক্রিয় যাচাই ও প্রয়োগ।

GitOps: Git-কে সত্যের উৎস ধরে অবকাঠামো পরিচালনার পদ্ধতি; PR-এর মাধ্যমে পরিবর্তন হয়।

Secret Manager: গোপন মান নিরাপদে সংরক্ষণ ও পড়ার সেবা; যেমন AWS Secrets Manager, HashiCorp Vault।

Vault: HashiCorp-এর secret ব্যবস্থাপনার টুল; dynamic secret ও encryption দেয়।

Encryption at Rest: সংরক্ষিত ডেটা (যেমন state ফাইল) এনক্রিপ্ট করে রাখা।

Encryption in Transit: নেটওয়ার্কে ডেটা পাঠানোর সময় TLS দিয়ে এনক্রিপ্ট করা।

Least Privilege: কাজের জন্য যতটুকু অনুমতি দরকার, শুধু ততটুকু দেওয়ার নীতি।

Idempotency: একই কাজ বারবার করলেও ফল একই থাকার বৈশিষ্ট্য; Terraform apply বারবার চালালেও অবস্থা একই থাকে।

Desired State: কনফিগারেশনে ঘোষিত চূড়ান্ত অবস্থা; Terraform বাস্তব অবস্থাকে এর সাথে মেলায়।

Actual State: ক্লাউডে বর্তমানে বিদ্যমান অবকাঠামোর বাস্তব অবস্থা।

Dependency Graph: resource-গুলোর মধ্যে নির্ভরতার কাঠামো; Terraform এতে parallel কাজের ক্রম ঠিক করে।

Parallelism: Terraform একসাথে একাধিক resource তৈরি করার ক্ষমতা; -parallelism flag দিয়ে সীমা বদলানো যায়।

ForceNew: এমন argument যা বদলালে resource ধ্বংস করে নতুন তৈরি করতে হয়; যেমন instance-এর AMI।

Replacement: পুরোনো resource মুছে নতুন তৈরি করার প্রক্রিয়া; ForceNew argument বদলালে ঘটে।

Tag: resource চেনা ও খরচ ভাগ করার জন্য key-value লেবেল; যেমন Environment = "prod"।

Naming Convention: resource-এর নাম দেওয়ার নিয়ম; পরিবেশ, প্রকল্প ও উদ্দেশ্য নামে থাকলে পরিচালনা সহজ হয়।

Environment: dev, staging, prod-এর মতো আলাদা ব্যবহারের স্তর; প্রতিটির আলাদা state ও variable থাকে।

Staging: production-এর মতো করে যাচাই করার পরিবেশ; release-এর আগে এখানে পরীক্ষা হয়।

Production: ব্যবহারকারীদের সেবা দেওয়া আসল পরিবেশ; এখানে apply-এ সবচেয়ে বেশি সতর্কতা দরকার।

Rollback: ভুল পরিবর্তনের পর আগের অবস্থায় ফেরার প্রক্রিয়া; Git history ও state ব্যাকআপ দিয়ে করা যায়।

Backup: state ও গুরুত্বপূর্ণ ডেটার অনুলিপি রাখা; ভুল হলে পুনরুদ্ধারে কাজে লাগে।

Versioning: S3 bucket-এ ফাইলের পুরোনো সংস্করণ রাখা; ভুল state ফিরিয়ে আনতে সাহায্য করে।

MFA Delete: S3-তে মুছে ফেলা আটকাতে MFA চাওয়া; backend bucket রক্ষায় ব্যবহৃত হয়।

Budget Alert: খরচ সীমা ছাড়ালে সতর্কবার্তা দেওয়ার ব্যবস্থা; cost explosion ধরতে সাহায্য করে।

Cost Estimation: apply-এর আগে আনুমানিক খরচ হিসাব করা; Infracost দিয়ে plan থেকে দেখা যায়।

Module Versioning: module-এর প্রতিটি release-এ সংস্করণ দেওয়া; version pin করে breaking change এড়ানো যায়।

Semantic Versioning: MAJOR.MINOR.PATCH নিয়মে সংস্করণ দেওয়া; major বদল মানে breaking change।

Changelog: প্রতিটি সংস্করণে কী বদলেছে তার তালিকা; upgrade-এর আগে পড়া উচিত।

Breaking Change: পুরোনো কনফিগারেশন আর না চলা এমন পরিবর্তন; major version-এ ঘটে।

Deprecation: কোনো argument বা resource ভবিষ্যতে সরিয়ে নেওয়া হবে বলে ঘোষণা; আগেই বিকল্প ব্যবহার করা উচিত।

Refactoring: বাস্তব resource না বদলে কোডের কাঠামো বদলানো; moved ব্লক ও state mv দিয়ে করা হয়।

Code Review: PR-এ অন্যের পর্যালোচনা; plan আউটপুট দেখে ভুল ধরা হয়।

Pull Request (PR): কোড পরিবর্তনের প্রস্তাব; CI-তে plan চালিয়ে পর্যালোচনা করা হয়।

Branching Strategy: Git-এ শাখা পরিচালনার নিয়ম; যেমন main শাখা থেকে production apply হয়।

Monorepo: একটি repository-তে একাধিক প্রকল্পের কোড রাখা; Terraform-এ environment আলাদা ডিরেক্টরিতে রাখা যায়।

Multi-cloud: একাধিক ক্লাউড (AWS + Azure) একসাথে ব্যবহার করা; Terraform একাধিক provider দিয়ে তা পারে।

Hybrid Cloud: নিজের ডেটা সেন্টার ও public cloud একসাথে ব্যবহার করা।

Cloud-init: VM প্রথমবার চালু হলে কনফিগারেশন চালানোর উপায়; provisioner-এর চেয়ে নির্ভরযোগ্য।

User Data: EC2 instance চালুর সময় দেওয়া স্ক্রিপ্ট; cloud-init দিয়ে কাজ চালানো যায়।

Load Balancer: ট্রাফিক একাধিক সার্ভারে ভাগ করার সেবা; Terraform-এ aws_lb দিয়ে তৈরি করা হয়।

Auto Scaling: চাহিদা অনুযায়ী সার্ভারের সংখ্যা বাড়ানো-কমানো; Terraform-এ autoscaling group দিয়ে করা হয়।

DNS: ডোমেইন নামকে IP-তে রূপান্তর করার ব্যবস্থা; যেমন Route 53।

TLS/SSL: নেটওয়ার্কে ডেটা এনক্রিপ্ট করার প্রোটোকল; সার্টিফিকেট দিয়ে পরিচয় যাচাই হয়।

Firewall: নেটওয়ার্ক ট্রাফিক নিয়ন্ত্রণ করার ব্যবস্থা; security group এর একটি রূপ।

CIDR Block: IP ঠিকানার পরিসর লেখার নিয়ম; যেমন 10.0.0.0/16।

cidrsubnet(): বড় CIDR থেকে সাবনেট CIDR গণনা করার Terraform function।

শব্দকোষ ৩ • Glossary 3

শব্দকোষ — তৃতীয় অংশ

শব্দকোষ — তৃতীয় অংশ

প্রতিটি শব্দের বাংলা ব্যাখ্যার সাথে ব্যবহারের প্রসঙ্গ দেওয়া আছে।

merge(): একাধিক map একত্র করার function; tag একত্র করতে ব্যবহৃত হয়।

lookup(): map থেকে নিরাপদে মান নেওয়ার function; key না থাকলে default দেয়।

coalesce(): প্রথম খালি নয় এমন মান ফেরত দেওয়ার function।

try(): প্রথম expression ব্যর্থ হলে বিকল্প চেষ্টা করার function।

jsonencode(): Terraform মানকে JSON string-এ রূপান্তর করার function; IAM policy-তে ব্যবহৃত হয়।

templatefile(): টেমপ্লেট ফাইল পড়ে variable বসিয়ে string তৈরি করার function।

toset(): list-কে set-এ রূপান্তর করার function; for_each-এ ব্যবহার করা যায়।

length(): তালিকা বা string-এর দৈর্ঘ্য দেওয়ার function।

format(): string-এ মান বসিয়ে সাজানোর function; যেমন নাম তৈরি করা।

State Migration: state এক backend থেকে অন্য backend-এ সরানো; init -migrate-state দিয়ে করা হয়।

State Backup: state ফাইলের ব্যাকআপ অনুলিপি; apply-এর আগে Terraform নিজে .backup ফাইল রাখে।

Audit Log: কে, কখন, কী পরিবর্তন করেছে তার রেকর্ড; Terraform Cloud-এ পাওয়া যায়।

Approval Gate: apply-এর আগে মানুষের অনুমোদন নেওয়ার ধাপ; production-এ ব্যবহৃত হয়।

Runbook: ঘটনা ঘটলে ধাপে ধাপে কী করতে হবে তার লিখিত নির্দেশনা।

Incident: সেবা ব্যাহত হওয়ার ঘটনা; case study-তে এর কারণ ও শিক্ষা বিশ্লেষণ করা হয়।

Postmortem: ঘটনার পর কারণ, প্রভাব ও শিক্ষা লেখা পর্যালোচনা; দোষ নয়, ব্যবস্থা ঠিক করাই লক্ষ্য।

Blameless Culture: ঘটনায় ব্যক্তিকে দোষ না দিয়ে ব্যবস্থার ত্রুটি ঠিক করার সংস্কৃতি।

Observability: লগ, মেট্রিক ও trace দিয়ে সিস্টেমের ভেতরের অবস্থা বোঝার ক্ষমতা।

High Availability: একটি অংশ নষ্ট হলেও সেবা চালু থাকার বৈশিষ্ট্য; একাধিক zone-এ resource ছড়িয়ে রাখা হয়।

Disaster Recovery: বড় বিপর্যয়ের পর সেবা ফিরিয়ে আনার পরিকল্পনা; ব্যাকআপ ও অন্য region-এ কপি থাকে।

Zero Downtime: পরিবর্তনের সময় সেবা বন্ধ না হওয়া; create_before_destroy দিয়ে সাহায্য করা যায়।

Blue-Green Deployment: নতুন সংস্করণ আলাদা পরিবেশে তৈরি করে ট্রাফিক বদলে দেওয়ার কৌশল।

Canary Deployment: নতুন সংস্করণ প্রথমে অল্প ব্যবহারকারীর ওপর পরীক্ষা করার কৌশল।

Immutable Infrastructure: চলমান সার্ভার বদল না করে নতুন সার্ভার তৈরি করে বদলে দেওয়ার নীতি।

Mutable Infrastructure: চলমান সার্ভারে সরাসরি বদল করার পদ্ধতি; drift-এর ঝুঁকি বেশি।

Configuration Drift: সময়ের সাথে কনফিগারেশন ও বাস্তব অবস্থার মধ্যে পার্থক্য তৈরি হওয়া।

Compliance: নিয়ম (যেমন encryption বাধ্যতামূলক) মেনে চলা; Policy as Code দিয়ে যাচাই করা হয়।

Governance: কে কী করতে পারবে, কী নিয়ম মানতে হবে — তার সামগ্রিক ব্যবস্থাপনা।

Least Privilege (আবার): কাজের জন্য যতটুকু দরকার শুধু ততটুকু অনুমতি দেওয়া; Terraform চালানো identity-তেও প্রযোজ্য।

Service Account: মানুষের বদলে অ্যাপ্লিকেশন বা Terraform-এর জন্য তৈরি পরিচয়; GCP-তে ব্যবহৃত হয়।

Managed Identity: Azure-এ resource-এর জন্য স্বয়ংক্রিয়ভাবে পরিচালিত পরিচয়; secret রাখতে হয় না।

Workload Identity: Kubernetes বা CI থেকে ক্লাউডে পরিচয় দেওয়ার নিরাপদ উপায়; দীর্ঘমেয়াদি key ছাড়া কাজ করে।

OIDC: OpenID Connect; CI থেকে ক্লাউডে স্বল্পমেয়াদি পরিচয় দেওয়ার প্রোটোকল।

Terraform Cloud: HashiCorp-এর পরিচালিত Terraform সেবা; remote state, run, policy ও দলগত কাজ দেয়।

Terraform Enterprise: নিজের পরিবেশে চালানো Terraform Cloud-এর সংস্করণ; বড় প্রতিষ্ঠানে ব্যবহৃত হয়।

Sentinel Policy: Terraform Cloud-এ নীতিমালা লেখার HashiCorp-এর ভাষা।

Cost Center: খরচ কোন দল বা প্রকল্পের, তা ভাগ করার হিসাব; tag দিয়ে চিহ্নিত করা হয়।

Resource Tagging: resource-এ tag লাগিয়ে মালিক, পরিবেশ ও খরচ চিহ্নিত করা।

Naming Prefix: সব resource-এর নামের শুরুতে পরিবেশ বা প্রকল্পের অংশ রাখা; যেমন prod-web-01।

Input Validation: variable-এর মান গ্রহণের আগে শর্ত দিয়ে যাচাই করা।

Output Sensitivity: output-এ sensitive = true দিয়ে মান গোপন রাখা।

Backend Configuration: backend ব্লকে state সংরক্ষণের সেটিং; যেমন bucket, key, region।

Partial Backend Configuration: backend-এর কিছু মান -backend-config flag দিয়ে দেওয়া; secret কোডে লেখা এড়ানো যায়।

State Pull/Push: remote state ডাউনলোড (pull) বা আপলোড (push) করার কমান্ড; জরুরি পুনরুদ্ধারে ব্যবহৃত হয়।

Workspace Selection: বর্তমান workspace বেছে নেওয়া; terraform workspace select দিয়ে করা হয়।

Plan-Only Run: apply না করে শুধু plan চালানো; CI-তে পর্যালোচনার জন্য ব্যবহৃত হয়।

Speculative Plan: PR-এ apply না করে শুধু পরিকল্পনা দেখানো; Terraform Cloud-এ ব্যবহৃত হয়।

Run Trigger: একটি workspace-এর apply শেষে অন্য workspace-এ run শুরু করা; Terraform Cloud-এর বৈশিষ্ট্য।

Variable Set: Terraform Cloud-এ একাধিক workspace-এ একই variable দেওয়ার উপায়।

Module Registry (Private): প্রতিষ্ঠানের ভেতরে শুধু নিজেদের module রাখার registry।

Git Source: Git repository থেকে module নেওয়ার উপায়; ?ref দিয়ে সংস্করণ নির্দিষ্ট করা যায়।

Local Module: একই repository-এর ভেতরে ./modules/... path দিয়ে ব্যবহার করা module।

Composition: ছোট ছোট module জোড়া দিয়ে বড় অবকাঠামো তৈরি করা।

Abstraction: জটিল বিবরণ লুকিয়ে সহজ interface দেওয়া; module-এর মূল উদ্দেশ্য।

Reusability: একই কোড বারবার ব্যবহার করার বৈশিষ্ট্য; module ও variable দিয়ে অর্জন করা যায়।

Dry Run: আসল পরিবর্তন না করে পরিকল্পনা যাচাই করা; terraform plan এর একটি রূপ।

Smoke Test: apply-এর পর মৌলিকভাবে সেবা চালু আছে কি না যাচাই করা।

Integration Test: একাধিক অংশ একসাথে কাজ করছে কি না যাচাই করা; Terratest দিয়ে করা যায়।

Unit Test (Terraform): একটি module-এর যুক্তি আলাদাভাবে যাচাই করা; terraform test দিয়ে করা যায়।

Mock Provider: আসল ক্লাউড ছাড়া পরীক্ষার জন্য নকল provider; terraform test-এ ব্যবহৃত হয়।

Override File: .tftest.hcl-এ mock বা override দিয়ে পরীক্ষার মান বদলানো।

Static Analysis: কোড না চালিয়ে যাচাই করা; Checkov, TFLint এর উদাহরণ।

Linting: কোডের শৈলী ও সাধারণ ভুল যাচাই করা; terraform fmt ও TFLint দিয়ে করা যায়।

Formatting: কোডের বিন্যাস একরকম করা; terraform fmt দিয়ে করা হয়।

Documentation (Module): module-এর README-তে input, output ও উদাহরণ লেখা; ব্যবহারকারীর জন্য জরুরি।

README: প্রকল্পের পরিচিতি ও ব্যবহারের নির্দেশনা লেখা ফাইল।

CHANGELOG: সংস্করণভেদে পরিবর্তনের তালিকা।

Semantic Release: commit বার্তা থেকে স্বয়ংক্রিয়ভাবে সংস্করণ দেওয়ার পদ্ধতি।

কমান্ড রেফারেন্স • Command Reference

প্রয়োজনীয় Terraform কমান্ড

প্রয়োজনীয় Terraform কমান্ড

কমান্ড	কাজ	উদাহরণ
`terraform init`	backend ও provider প্রস্তুত করা	`terraform init`
`terraform fmt`	কোডের বিন্যাস ঠিক করা	`terraform fmt -recursive`
`terraform validate`	syntax ও reference যাচাই করা	`terraform validate`
`terraform plan`	পরিবর্তনের পূর্বাভাস দেখা	`terraform plan -out=tfplan`
`terraform apply`	পরিবর্তন প্রয়োগ করা	`terraform apply tfplan`
`terraform destroy`	সব managed resource মুছে দেওয়া	`terraform destroy`
`terraform show`	state বা plan-এর বিবরণ দেখা	`terraform show tfplan`
`terraform output`	ঘোষিত output-এর মান দেখা	`terraform output public_ip`
`terraform state list`	state-এর resource তালিকা দেখা	`terraform state list`
`terraform state show`	একটি resource-এর state দেখা	`terraform state show aws_instance.web`
`terraform state mv`	state-এ resource-এর ঠিকানা বদলানো	`terraform state mv aws_instance.a aws_instance.b`
`terraform state rm`	state থেকে resource সরানো	`terraform state rm aws_instance.old`
`terraform state pull / push`	remote state ডাউনলোড/আপলোড করা	`terraform state pull > backup.tfstate`
`terraform import`	বিদ্যমান resource state-এ আনা	`terraform import aws_instance.web i-0123456789`
`terraform graph`	dependency গ্রাফ দেখা	`terraform graph | dot -Tpng > graph.png`
`terraform workspace`	workspace তৈরি/বদল/তালিকা করা	`terraform workspace new prod`
`terraform providers`	ব্যবহৃত provider ও সংস্করণ দেখা	`terraform providers`
`terraform version`	Terraform ও provider সংস্করণ দেখা	`terraform version`
`terraform test`	লেখা টেস্ট চালানো	`terraform test`
		`terraform force-unlock <lock-id>`

`terraform force-unlock`	আটকে থাকা lock খোলা (শুধু নিশ্চিত হলে)	
`terraform refresh-only`	বাস্তব অবস্থা পড়ে state মেলানো (plan -refresh-only)	`terraform plan -refresh-only`

> **মনে রাখবেন:** `force-unlock` ও `state push` বিপজ্জনক কমান্ড — শুধু নিশ্চিত হয়ে, ব্যাকআপ নিয়ে ব্যবহার করুন।

HCL রেফারেন্স • HCL Syntax Reference

HCL সিনট্যাক্স রেফারেন্স

HCL সিনট্যাক্স রেফারেন্স

মৌলিক ব্লক

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {
    bucket = "my-tf-state"
    key    = "prod/terraform.tfstate"
    region = "ap-south-1"
  }
}

provider "aws" {
  region = var.region
}

resource "aws_instance" "web" {
  ami           = var.ami
  instance_type = "t3.micro"
  tags = {
    Name = "web-01"
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]
}
```

Variable, Output ও Locals

```
variable "instance_count" {
  type        = number
  default     = 2
  description = "কতটি instance তৈরি হবে"
  validation {
    condition     = var.instance_count > 0
    error_message = "instance_count ০-এর বেশি হতে হবে।"
  }
}

variable "tags" {
```

```
  type    = map(string)
  default = {}
}

output "public_ip" {
  value     = aws_instance.web.public_ip
  sensitive = false
}

locals {
  name_prefix = "${var.project}-${var.environment}"
  common_tags = merge(var.tags, { Environment = var.environment })
}
```

টাইপ

ধরন	উদাহরণ
string	`"ap-south-1"`
number	`3`
bool	`true`
list(string)	`["a", "b"]`
set(string)	`toset(["a", "b"])`
map(string)	`{ Name = "web" }`
object	`object({ name = string, size = number })`

Meta-Argument ও Dynamic Block

```
resource "aws_instance" "web" {
  count         = var.instance_count
  ami           = var.ami
  instance_type = "t3.micro"
  depends_on    = [aws_security_group.web]

  lifecycle {
    prevent_destroy       = false
    create_before_destroy = true
    ignore_changes        = [tags]
  }
}

resource "aws_security_group" "web" {
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = "tcp"
```

```
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}

module "vpc" {
  source  = "./modules/vpc"
  version = "1.2.0"
  cidr    = "10.0.0.0/16"
}
```

বহুল ব্যবহৃত Function

Function	কাজ	উদাহরণ
`length()`	দৈর্ঘ্য	`length(["a", "b"]) # 2`
`merge()`	map একত্র করা	`merge({a=1}, {b=2})`
`format()`	string সাজানো	`format("%s-%s", "web", "01")`
`lookup()`	map থেকে নিরাপদে মান নেওয়া	`lookup(var.tags, "Env", "dev")`
`coalesce()`	প্রথম খালি নয় এমন মান	`coalesce(var.a, var.b, "default")`
`try()`	ব্যর্থ হলে বিকল্প চেষ্টা	`try(var.obj.name, "unknown")`
`jsonencode()`	JSON string-এ রূপান্তর	`jsonencode({Version="2012-10-17"})`
`templatefile()`	টেমপ্লেটে মান বসানো	`templatefile("user-data.sh", {name="web"})`
`toset()`	list-কে set করা	`toset(["a", "b"])`
`cidrsubnet()`	CIDR থেকে সাবনেট বের করা	`cidrsubnet("10.0.0.0/16", 8, 1)`

কাজের ধারা • Workflow Reference

কাজের ধারা রেফারেন্স

কাজের ধারা (Workflow) রেফারেন্স

Plan / Apply ধারা

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
# plan পড়ুন: create / update / destroy / replace যাচাই করুন
terraform apply tfplan
terraform output
```

Import ধারা

```
# ১. কনফিগারেশনে খালি resource ব্লক লিখুন
# ২. import করুন
terraform import aws_instance.web i-0123456789
# ৩. terraform show দিয়ে বাস্তব মান দেখে কনফিগারেশন মেলান
terraform plan   # "No changes" না হওয়া পর্যন্ত মেলান
```

Module কাঠামোর নমুনা

```
modules/vpc/
  main.tf        # resource ব্লক
  variables.tf   # input variable
  outputs.tf     # output
  README.md      # ব্যবহারের নির্দেশনা
  versions.tf    # required_providers
```

Backend কনফিগারেশনের উদাহরণ

```
# S3 + DynamoDB (AWS)
terraform {
  backend "s3" {
    bucket         = "my-tf-state"
    key            = "prod/terraform.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "tf-state-lock"
    encrypt        = true
  }
}

# azurerm (Azure)
terraform {
```

```
  backend "azurerm" {
    resource_group_name  = "tfstate-rg"
    storage_account_name = "tfstate123"
    container_name       = "tfstate"
    key                  = "prod.terraform.tfstate"
  }
}

# gcs (Google Cloud)
terraform {
  backend "gcs" {
    bucket = "my-tf-state"
    prefix = "prod"
  }
}
```

যাচাই কমান্ড

```
terraform fmt -check -recursive   # বিন্যাস যাচাই
terraform validate                # syntax যাচাই
terraform plan -refresh-only      # drift যাচাই
terraform test                    # লেখা টেস্ট চালানো
tflint                            # lint যাচাই
checkov -d .                      # নিরাপত্তা যাচাই
```

সাধারণ ত্রুটি → কারণ → সমাধান

ত্রুটি	সম্ভাব্য কারণ	সমাধান
Error acquiring the state lock	আরেকটি apply চলছে বা lock আটকে আছে	চলমান apply যাচাই করে অপেক্ষা করুন; নিশ্চিত হলে force-unlock
No changes / অপ্রত্যাশিত destroy	নাম, key বা module বদলেছে	Git diff দেখুন; moved ব্লক বা state mv ব্যবহার করুন
Provider version conflict	constraint ও lock ফাইলের অমিল	constraint ঠিক করুন বা init -upgrade চালান
Access denied (backend)	credential বা IAM permission ভুল	সঠিক credential ও bucket policy যাচাই করুন
Invalid reference	ভুল resource নাম বা attribute	validate-এর বার্তা পড়ে reference ঠিক করুন
For_each key বদলে recreation	অস্থিতিশীল key ব্যবহার	স্থিতিশীল key ব্যবহার করুন; state mv দিয়ে মেলান

চেকলিস্ট • Checklists

চেকলিস্ট

চেকলিস্ট

নিরাপত্তা চেকলিস্ট

☐ Secret কোডে বা Git-এ নেই — secret manager থেকে পড়া হয়। ☐ State backend-এ encryption চালু আছে। ☐ State-এর access শুধু প্রয়োজনীয় ব্যক্তি/role-এর। ☐ Security group-এ 0.0.0.0/0 দিয়ে SSH খোলা নেই। ☐ S3 bucket public নয়। ☐ .gitignore-এ *.tfstate ও .terraform/ আছে। ☐ Sensitive output-এ sensitive = true আছে।

Production IaC চেকলিস্ট

☐ Provider ও module সংস্করণ pin করা আছে। ☐ Remote backend ও locking চালু আছে। ☐ গুরুত্বপূর্ণ resource-এ prevent_destroy আছে। ☐ ব্যাকআপ ও পুনরুদ্ধারের পরিকল্পনা আছে। ☐ Plan PR-এ পর্যালোচনা হয়। ☐ Apply-এর আগে অনুমোদন নেওয়া হয়।

Apply-এর আগের চেকলিস্ট

☐ সঠিক workspace ও backend যাচাই করা হয়েছে। ☐ Plan পড়া হয়েছে — destroy/replace আছে কি না দেখা হয়েছে। ☐ Resource-এর সংখ্যা ও আকার যাচাই করা হয়েছে। ☐ খরচের প্রভাব দেখা হয়েছে। ☐ ব্যাকআপ আছে কি না যাচাই করা হয়েছে।

ব্যাকআপ ও পুনরুদ্ধার চেকলিস্ট

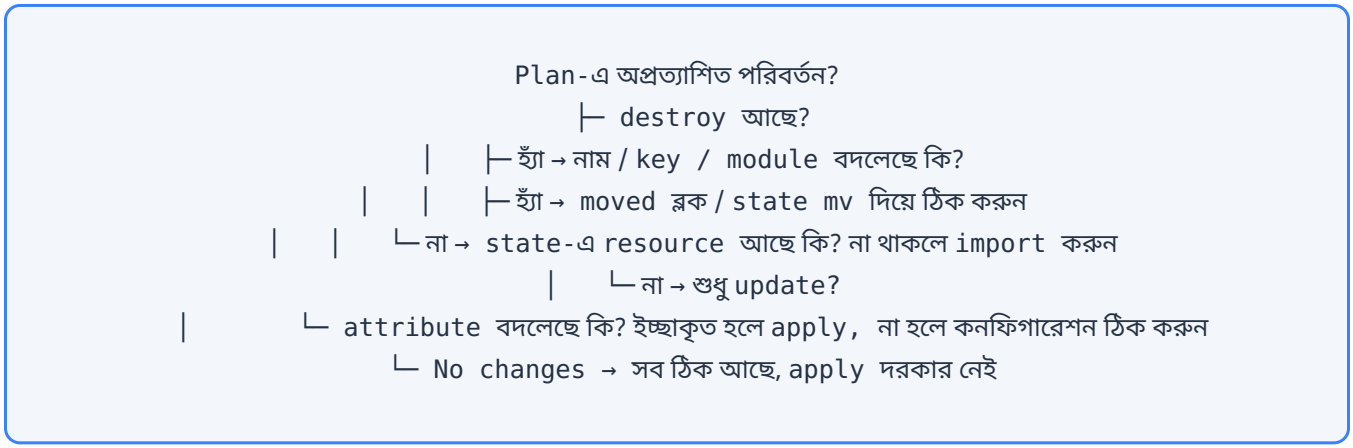
☐ State-এর ব্যাকআপ (versioning বা নিয়মিত copy) আছে। ☐ Backend bucket-এ versioning চালু আছে। ☐ State পুনরুদ্ধারের ধাপ লিখিত আছে (state pull/push)। ☐ গুরুত্বপূর্ণ ডেটার ব্যাকআপ পরীক্ষা করা হয়েছে।

সিদ্ধান্ত-বৃক্ষ • Troubleshooting Decision Trees

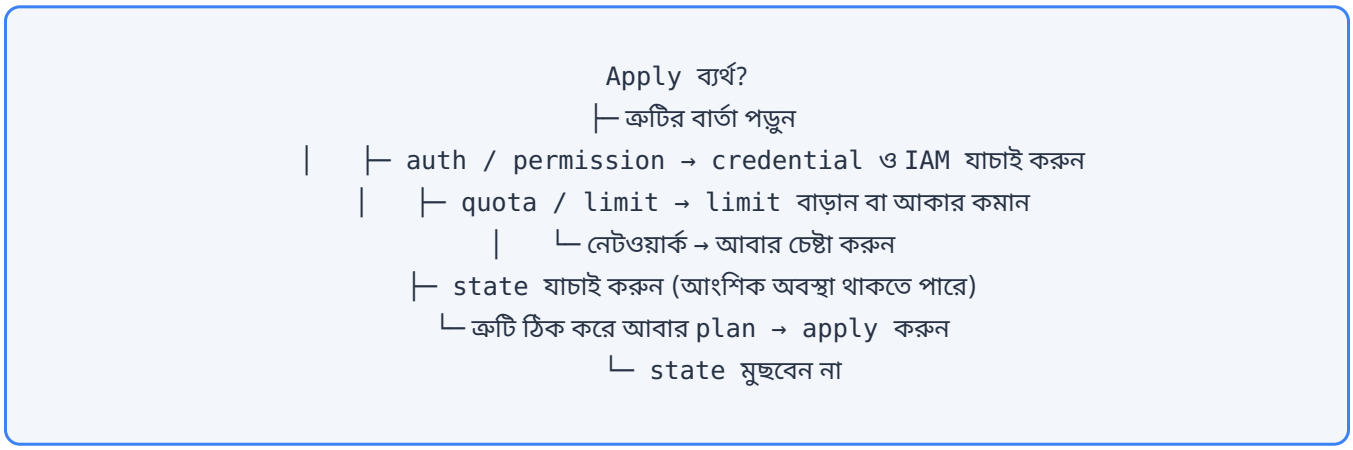
সমস্যা নির্ণয়ের সিদ্ধান্ত-বৃক্ষ

সমস্যা নির্ণয়ের সিদ্ধান্ত-বৃক্ষ

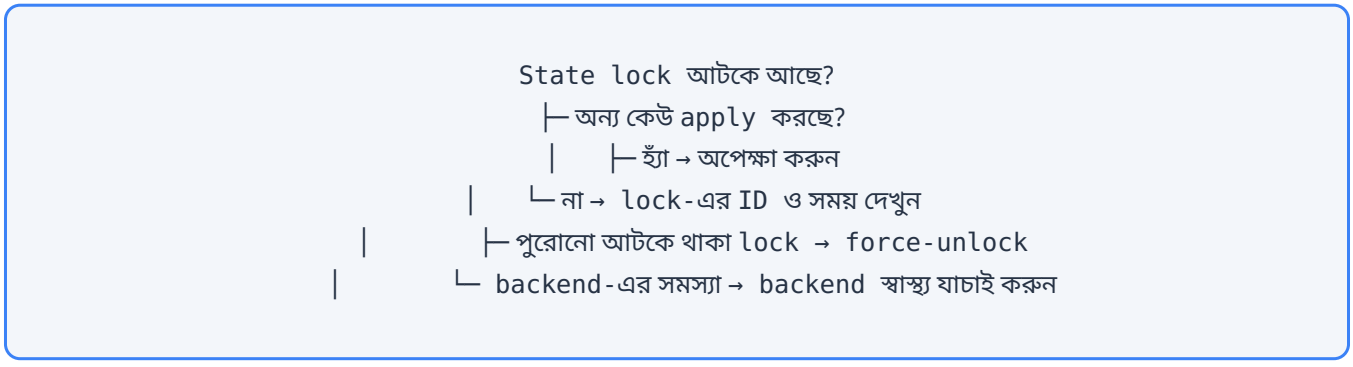
Plan-এ অপ্রত্যাশিত পরিবর্তন দেখালে



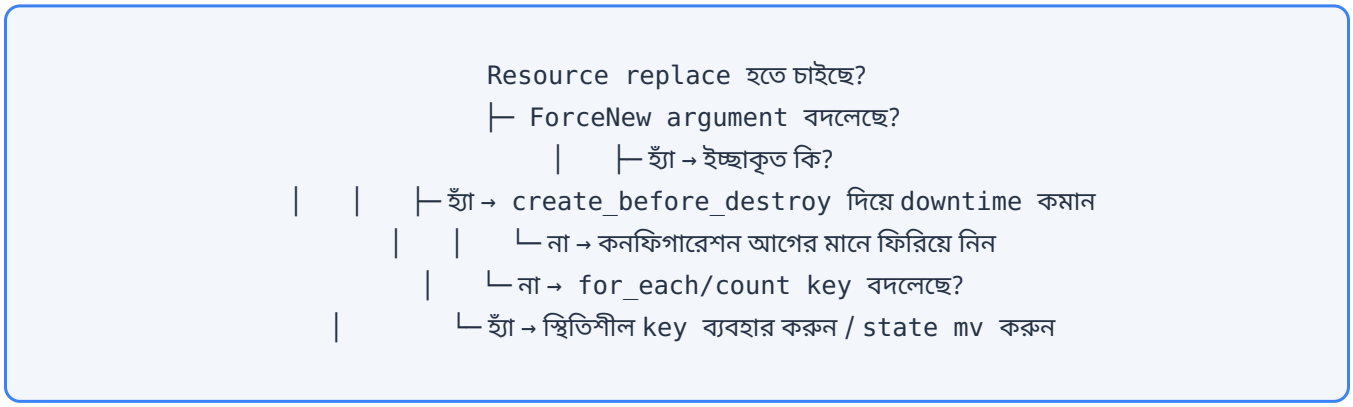
Apply ব্যর্থ হলে



State Lock আটকে গেলে



Resource পুনরায় তৈরি হতে চাইলে



দক্ষতা যাচাইপত্র • Competency Tracking Sheet

ব্যক্তিগত দক্ষতা যাচাইপত্র

ব্যক্তিগত দক্ষতা যাচাইপত্র

প্রতিটি দক্ষতায় নিজের স্তর লিখুন — ০ = জানি না, ১ = ধারণা আছে, ২ = নিজে করে দেখেছি, ৩ = অন্যকে শেখাতে পারি। নিয়মিত হালনাগাদ করুন।

দক্ষতা	স্তর (০–৩)	অনুশীলনের তারিখ	মন্তব্য
IaC ধারণা ও declarative পদ্ধতি			
HCL লেখা			
Variable / output / locals			
Resource ও data source			
State ও backend			
State locking ও drift			
Plan / apply / destroy			
Lifecycle নিয়ম			
count / for_each			
Dynamic block ও function			
Module তৈরি ও ব্যবহার			
Module versioning			
Import ও migration			
Workspace			
Provider constraint ও lock ফাইল			
AWS provisioning			
Azure provisioning			
GCP provisioning			
Secret ও নিরাপত্তা			
Policy as Code (OPA/Checkov)			
CI/CD-এ Terraform			
Testing (terraform test / Terratest)			

Cost estimation			
Troubleshooting ও পুনরুদ্ধার			

রেফারেন্স • Official References

সরকারি রেফারেন্স

সরকারি রেফারেন্স

নিচের তালিকায় শুধু যাচাই করা সরকারি ঠিকানা দেওয়া আছে। কোনো ঠিকানা না খুললে সার্চ ইঞ্জিনে প্রকল্পের নাম লিখে সরকারি সাইট খুঁজুন।

- HashiCorp Terraform ডকুমেন্টেশন — https://developer.hashicorp.com/terraform
- Terraform ভাষা (Language) ডকুমেন্টেশন — https://developer.hashicorp.com/terraform/language
- Terraform CLI ডকুমেন্টেশন — https://developer.hashicorp.com/terraform/cli
- HCL ডকুমেন্টেশন — https://developer.hashicorp.com/terraform/language/syntax
- Terraform Registry — https://registry.terraform.io
- AWS Provider ডকুমেন্টেশন — https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- AWS ডকুমেন্টেশন — https://docs.aws.amazon.com
- Azure Provider (azurerm) ডকুমেন্টেশন — https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs
- Microsoft Azure ডকুমেন্টেশন — https://learn.microsoft.com/azure/
- Google Cloud Provider ডকুমেন্টেশন — https://registry.terraform.io/providers/hashicorp/google/latest/docs
- Google Cloud ডকুমেন্টেশন — https://cloud.google.com/docs
- IBM Cloud Provider ডকুমেন্টেশন — https://registry.terraform.io/providers/IBM-Cloud/ibm/latest/docs
- IBM Cloud ডকুমেন্টেশন — https://cloud.ibm.com/docs
- Open Policy Agent (OPA) — https://www.openpolicyagent.org/docs/
- TFLint — https://github.com/terraform-linters/tflint
- Checkov — https://www.checkov.io